Fault Diagnosis of BLDC Motors Using CNN

Convolutional Neural Networks (CNNs) are a specialized type of neural network primarily used for analyzing visual data.

Presented By: Anshul

Agenda

01 Overview of BLDC Motors

02 CONCEPT OF CNN(THEORY)

03 Mathematical Foundation of CNNs

04 Experimental Setup

05 Model training

06 Final Results

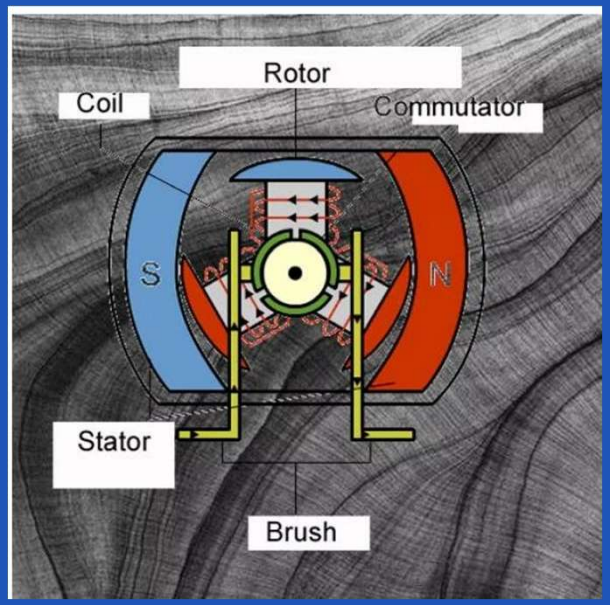
Overview of BLDC Motors

BLDC motors use a permanent magnet rotor and stationary coils on the stator, eliminating brushes and commutators. This design offers high efficiency, low noise, and long life, making them popular in automotive, aerospace, and industrial applications.

•Less maintenance and noise

•High speed and thermal performance

•Smaller, lighter geometry



Study Objectives

BLDC motor fault diagnosis using preprocessing, modeling, evaluation, and robust validation.

Develop a fault diagnosis system using traditional and deep learning models.

Key goals include converting sensor data to images, comparing classifiers, applying 5-fold cross-validation, and exploring noise robustness and explainability.

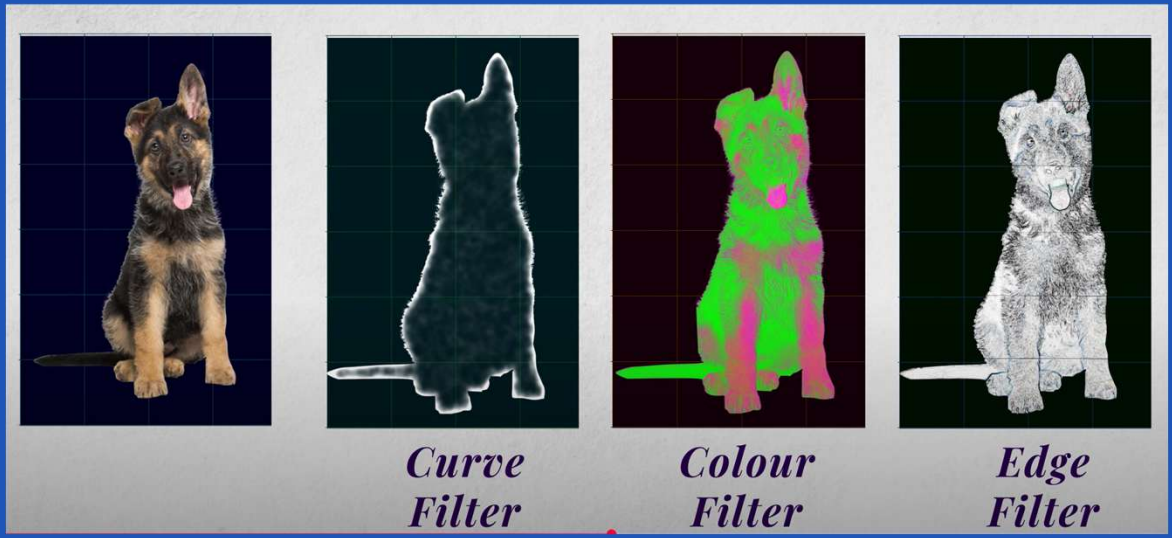
CONCEPT OF CNN (THEORY)

CONCEPT (THEORY)

1. Convolutional Layer

This is the heart of a CNN. It performs the **convolution** operation, which involves a small matrix called a **filter** (or kernel) sliding over the input image

How it works: The filter moves across the image, performing an element-wise multiplication with the portion of the image it's currently on, and then summing the results to create a single pixel in a new image called a **feature map**.



2. The Pooling Operation

Pooling layers perform downsampling, which reduces the dimensions of the feature maps. While there are different types, such as average pooling, **max pooling** is the most common

Mathematically, for a patch of size p×p in the feature map S, the max pooling operation P is:

$$P(i,j) = \max_{m,n \in \text{patch}} S(i+m, j+n)$$

This operation preserves the most prominent features detected by the convolutional layer

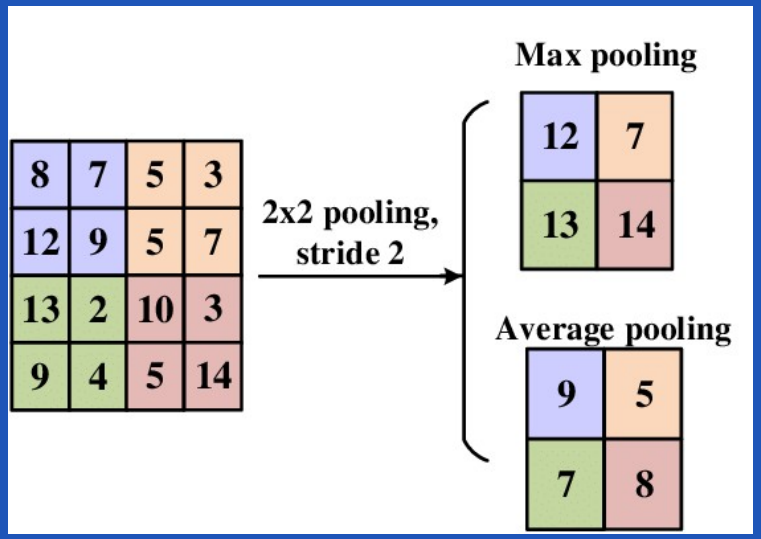
CONCEPT (THEORY)

2. Pooling Layer

A **Pooling Layer** follows a convolutional layer to **reduce the spatial size** of the feature map.
Its main **Purpose** is **downsampling** the data to:

1.Decrease computational load.
2.Increase robustness to small shifts in feature position.

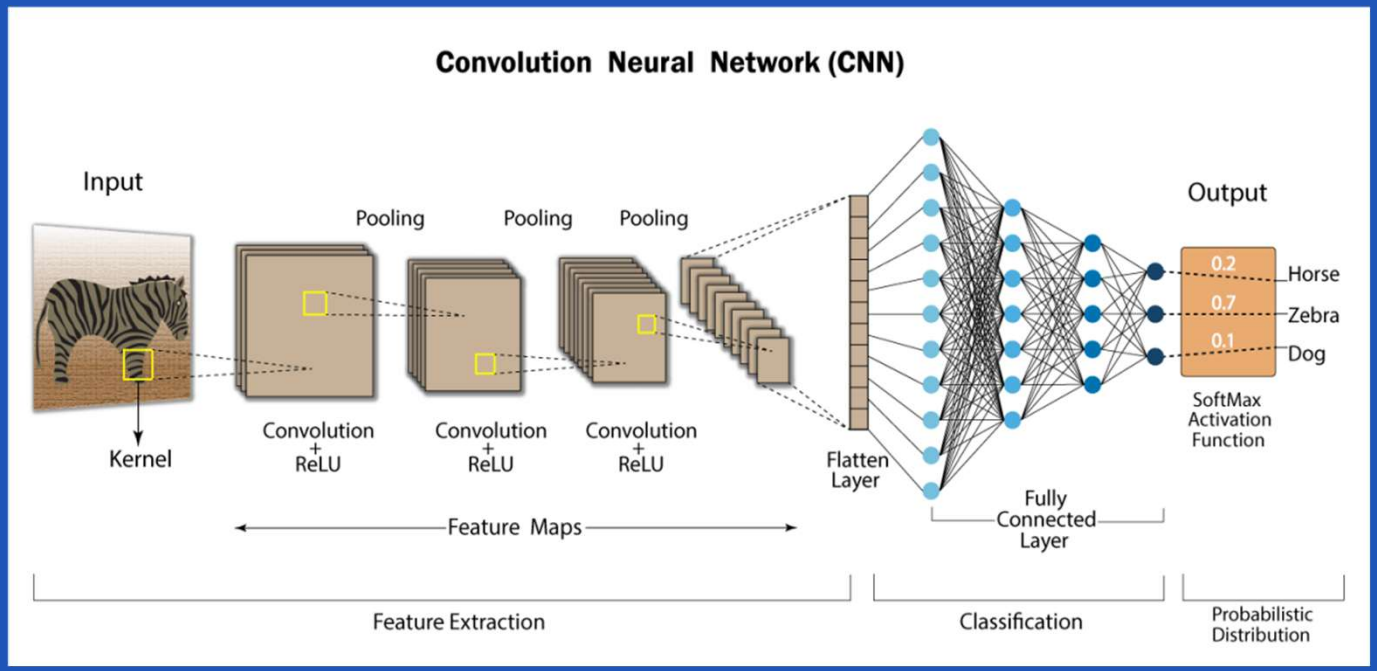
The most common **Type** is **Max Pooling**,
which selects the **maximum value** from a small region,
effectively retaining the most prominent feature
while making the data representation more compact.



CONCEPT (THEORY)

3. Fully Connected Layer

The final layers of a CNN are fully connected, similar to traditional neural networks. After feature extraction, the output is flattened into a vector and passed through these layers. They use the learned features to perform classification, mapping them to outputs like “Zebra” or “Dog.”



Mathematical Foundation of CNNs

01 Convolution Operation (Feature Extraction)

02 The Pooling Operation

03 Activation Functions

04 Training: Backpropagation and Gradient Descent

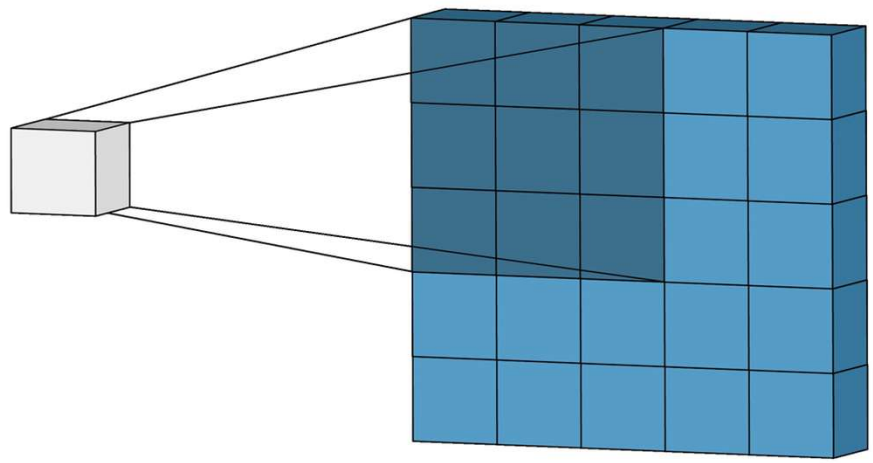
Convolution Operation (Feature Extraction)

The core of a CNN is the **convolution** operation. It's a mathematical process that combines two functions to produce a third function.

In a CNN, this is a discrete convolution performed on the input data and a small matrix called a **filter** (or kernel).

Given a 2D input image I and a 2D filter K, the convolution operation to produce a feature map S is defined as:

Convolution Operation (Feature Extraction)



$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i-m, j-n)K(m,n)$$

I is the input image.
K is the filter.
S is the resulting feature map.

3. Activation Functions

After convolution, a **non-linear activation function** is applied to the feature map. This is crucial because it introduces **non-linearity**, allowing the model to learn complex, non-linear patterns.

The most popular activation function is the Rectified Linear Unit (ReLU), defined as:

$$f(x) = \max(0, x)$$

This simple function sets any negative input to zero while leaving positive inputs unchanged.

4. Training: Backpropagation and Gradient Descent

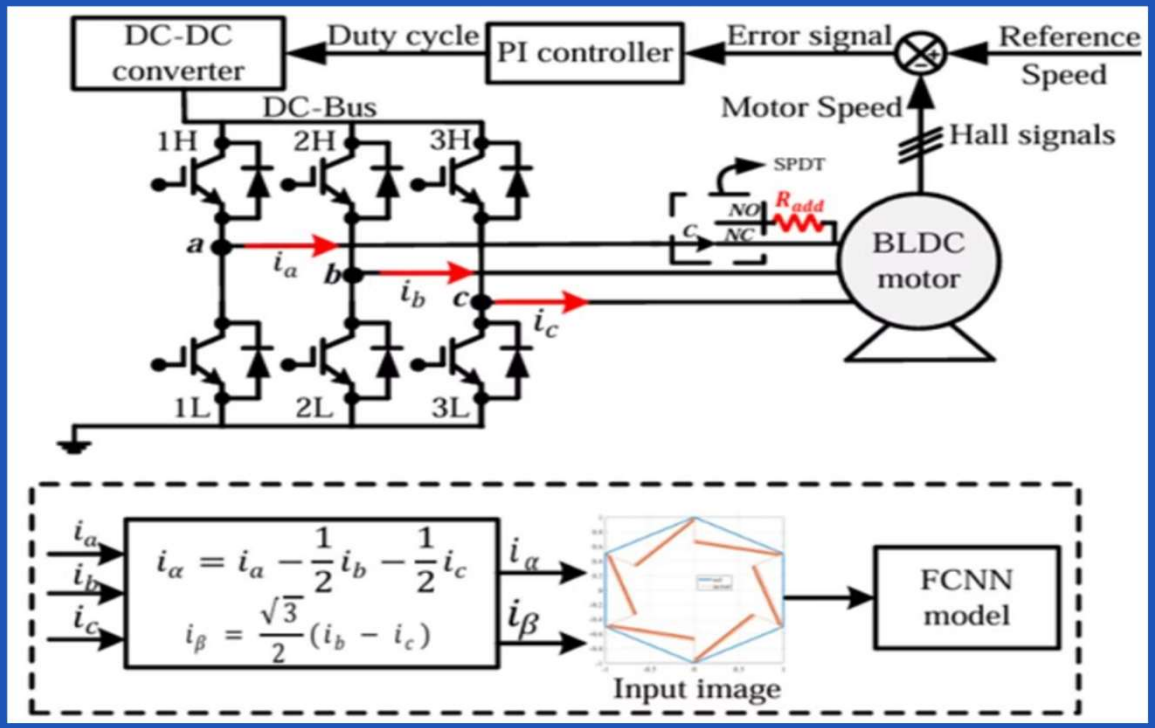
The network is trained by adjusting weights and biases to minimize a loss function, using backpropagation and gradient descent.

Gradient Descent: This is an iterative optimization algorithm that finds the minimum of a function. The update rule for a parameter (weight) theta is:

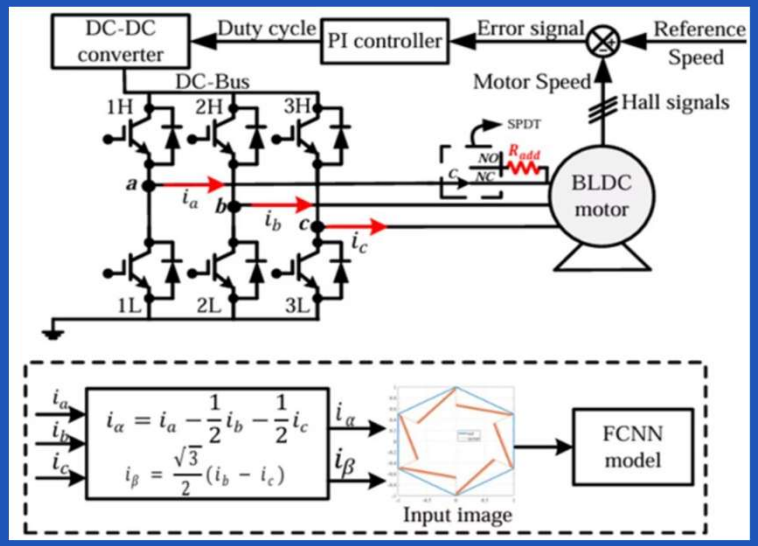
$$\theta_{new} = \theta_{old} - \alpha \nabla_\theta J(\theta)$$

The **Gradient Descent** rule updates the **new parameters** (θnew) by taking the **old parameters** (θold) and moving them a step (α) in the **opposite direction** (- sign) of the **gradient** ($\nabla\theta J(\theta)$) of the loss function, which minimizes the error.

Experimental Setup



The dataset includes images from experiments simulating various BLDC motor faults. The circuit arrangement allows fault injection and data collection under controlled conditions.



Fault Types

- Stator winding faults: short/open circuits
- Rotor faults: misalignment, rubbing
- Sensor faults: position and current sensor
- Phase faults: inverter and ground faults
- Mechanical faults: bearing failures, shaft coupling issues

Model Training

1. Image Input From Dataset

2. CNN Architecture for Feature Extraction

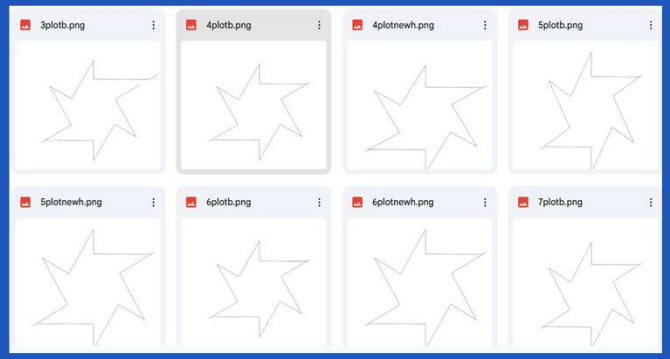
3. Training Using 5-Fold Cross-Validation

4. Improving Accuracy Through Iterations (Epochs)

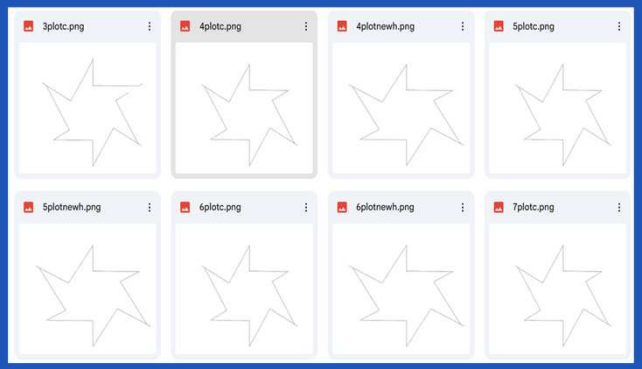
1. Image Input From Dataset

Fault A

Fault B

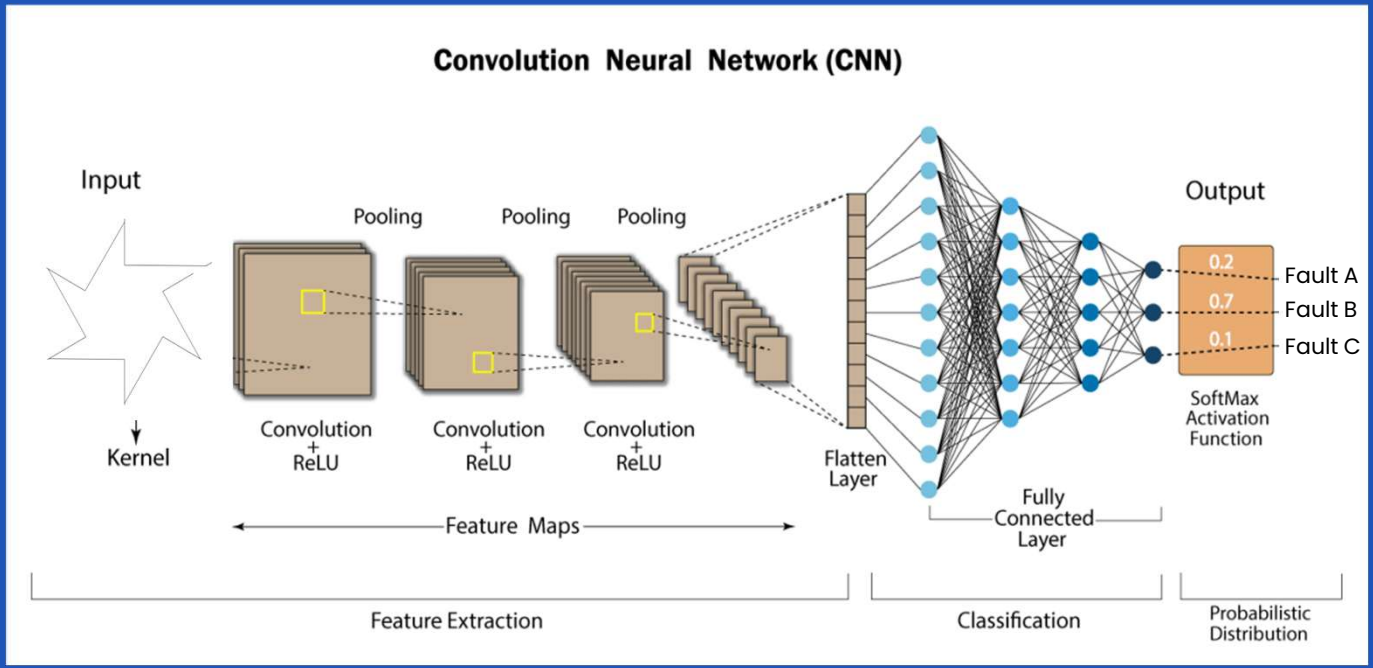


Fault C

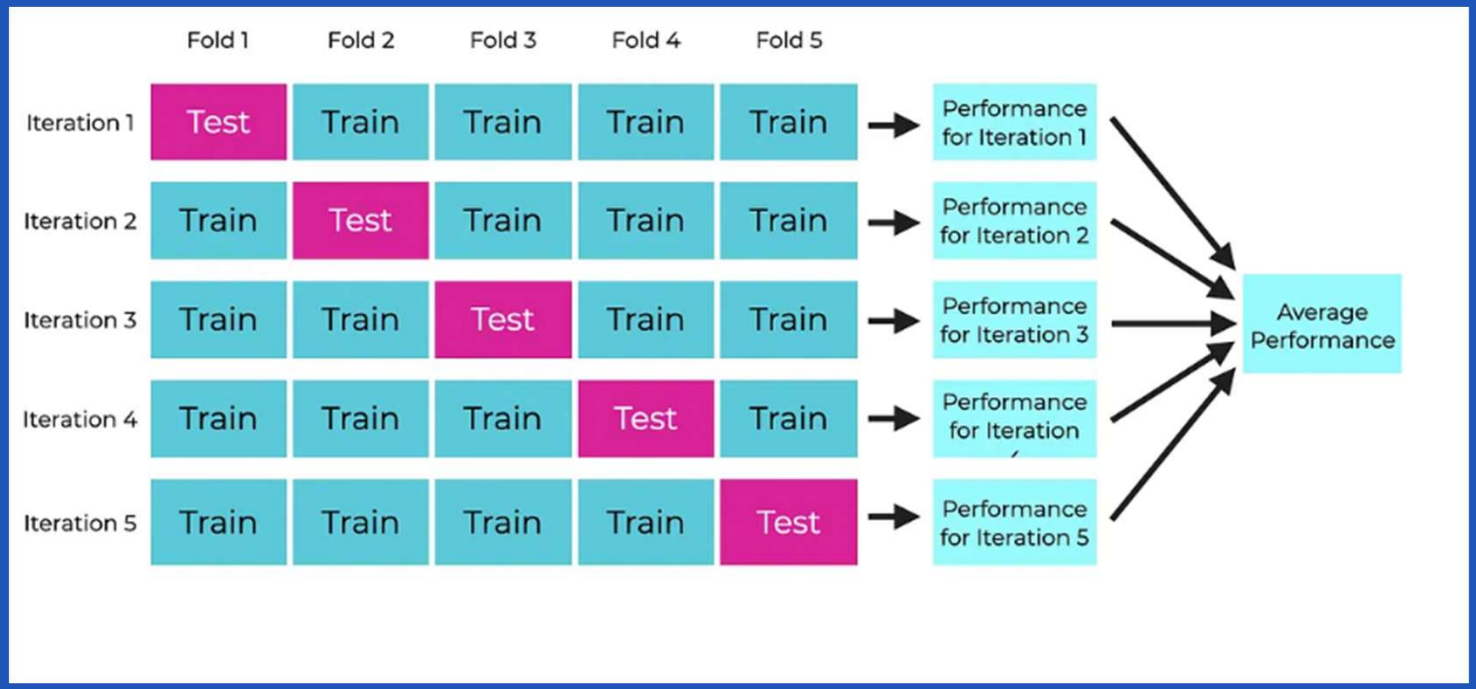


Healthy

CNN Architecture for Feature Extraction



Training Using 5-Fold Cross-Validation



Results

```
CROSS-VALIDATION SUMMARY
============================================================
Mean Accuracy Across All Folds: 94.70%
Standard Deviation: 5.20%
Best Performing Fold Accuracy: 100.00%
Best Model Saved At: model_weights_OC2.h5

Individual Fold Accuracies:
  Fold 1: 99.50%
  Fold 2: 100.00%
  Fold 3: 96.50%
  Fold 4: 91.00%
  Fold 5: 86.50%
```

RESULT

My CNN model initially gave around 90% accuracy, but after making some changes and tuning a few parameters, the accuracy increased to 94.70% average in 5-fold cross-validation, with Fold 2 hitting 100% accuracy. I'm still trying out more ways to push it further, like adjusting the learning rate, adding data augmentation, and increasing the number of epochs for better feature learning.

Thank You